

GUIA DE ESTUDO

Backend do Sistema de Controle de Estoque

Resumo técnico, roteiro de demonstração e perguntas prováveis da banca

Equipe 02 — Edypho, Lucas e Nicollas

1. Como usar este guia

Este material foi feito a partir do código atual do projeto. Estude primeiro a visão geral e o fluxo de uma movimentação. Depois treine as perguntas em voz alta, tentando responder em 20 a 40 segundos.

O mínimo que todo integrante deve saber:

- Explicar o caminho JavaScript → Flask → regra de negócio → SQLite → JSON.
- Explicar as três tabelas e seus relacionamentos.
- Demonstrar entrada, saída, cálculo do saldo e estoque mínimo.
- Explicar por que produto removido é inativado, e não apagado.
- Dizer como o sistema evita saldo negativo e dados inválidos.
- Mostrar como os testes usam bancos separados.

Resposta de abertura recomendada: “Nosso backend foi desenvolvido em Python com Flask. Ele recebe requisições do frontend em JavaScript, valida as regras de negócio, executa comandos parametrizados no SQLite e devolve respostas JSON. O sistema controla categorias, produtos, entradas, saídas, saldos, alertas e relatórios.”

2. Visão geral da arquitetura

FRONTEND	COMUNICAÇÃO	BACKEND	PERSISTÊNCIA
HTML + CSS + JavaScript	fetch() / HTTP / JSON	Python + Flask	SQLite (estoque.db)
Telas e interação	GET, POST e DELETE	Validação e regras	Categorias, produtos e movimentações

Fluxo de uma saída de estoque:

- O usuário informa quantidade e observação na tela de produtos.
- O JavaScript envia POST para /produtos/<id>/saida com um JSON.
- A API lê o JSON e chama movimentacoes.registrar_saida().
- O backend verifica produto ativo, quantidade inteira positiva e saldo disponível.
- Na mesma transação, grava a movimentação e atualiza a quantidade do produto.
- O commit confirma as duas alterações; se houver erro, rollback desfaz tudo.
- A API consulta e devolve o novo saldo em JSON.

3. Responsabilidade de cada arquivo

Arquivo	Responsabilidade	O que saber explicar
backend/api.py	Rotas Flask, JSON, códigos HTTP e entrega do frontend.	É a porta de entrada das requisições.
backend/database.py	Conexão, criação das tabelas e migração de colunas.	Ativa chaves estrangeiras e usa sqlite3.Row.
backend/produtos.py	Cadastro/listagem de categorias e produtos; inativação.	Valida tipos, valores e categoria ativa.
backend/movimentacoes.py	Entrada, saída, saldo, histórico, estoque baixo e dashboard.	Usa transação com commit/rollback.
backend/relatorios.py	Prepara inventário e movimentações em dicionários.	Calcula valor total e indicador de estoque baixo.
backend/exceptions.py	Exceções conhecidas do domínio.	Permite responder 400 ou 404 com mensagem clara.
backend/seed.py	Cria dados de informática para demonstração.	Evita repetir as movimentações do próprio seed.

4. Banco de dados e relacionamentos

Tabela	Campos importantes	Finalidade
categorias	id, nome, descricao, ativo, criado_em	Organizar produtos. O nome é único.
produtos	id, nome, categoria_id, preco, quantidade, estoque_minimo, ativo	Guardar o saldo atual e dados do produto.
movimentacoes	produto_id, tipo, quantidade, data_hora, saldo_anterior, saldo_atual	Manter o histórico de entradas e saídas.

Relacionamentos: uma categoria pode possuir vários produtos; um produto pode possuir várias movimentações. As chaves estrangeiras são categoria_id e produto_id.

Detalhes importantes do SQLite:

- sqlite3.Row permite acessar uma coluna por nome, por exemplo produto['quantidade'].
- PRAGMA foreign_keys = ON ativa a verificação das chaves estrangeiras em cada conexão.
- CREATE TABLE IF NOT EXISTS cria as tabelas somente quando ainda não existem.
- PRAGMA table_info e ALTER TABLE adicionam colunas novas sem apagar o banco antigo.
- Os valores do usuário são enviados ao SQL por ?, evitando concatenar entrada diretamente.

Pegadinha: os campos descricao, sku e fornecedor existem em produtos, mas a API atual de cadastro ainda não os recebe. Eles vieram da evolução da modelagem e podem ser implementados em uma versão futura.

5. Regras de negócio

Regra	Como o backend garante
Nome obrigatório	Verifica se é string e se não fica vazio após strip().
Categoria válida	O produto só é criado se a categoria existir e estiver ativa.
Preço não negativo	Aceita int/float, rejeita bool, texto e valor menor que zero.
Quantidade inteira	Rejeita decimal, bool, texto, zero ou negativo em movimentações.
Saldo não negativo	Uma saída maior que saldo_anterior gera EstoqueInsuficienteError.
Saldo automático	Entrada soma; saída subtrai; produto é atualizado na mesma transação.
Estoque mínimo	Alerta quando quantidade <= estoque_minimo.
Histórico preservado	DELETE da API faz UPDATE ativo = 0; não apaga fisicamente.

Exemplo numérico para explicar: produto com saldo 20. Uma saída de 8 grava saldo_anterior = 20, saldo_atual = 12 e atualiza produtos.quantidade para 12. Uma saída de 30 é recusada antes de qualquer gravação.

6. Transação, commit, rollback e fechamento

A movimentação altera duas informações que precisam permanecer consistentes: o histórico e o saldo do produto. Por isso, as duas operações usam a mesma conexão.

try:

```

INSERT INTO movimentacoes (...)
UPDATE produtos SET quantidade = ...
conn.commit()
except Exception:
    conn.rollback()
    raise
finally:
    conn.close()

```

Atomicidade: ou as duas alterações são confirmadas, ou nenhuma delas é mantida. Isso evita uma movimentação sem saldo atualizado ou um saldo alterado sem histórico.

7. Rotas da API e códigos HTTP

Método	Rota	Função	Sucesso
GET	/	Entrega index.html	200
GET	/categorias	Lista categorias ativas	200
POST	/categorias	Cadastra categoria	201
GET	/produtos	Lista produtos ativos	200
GET	/produtos/	Busca produto ativo	200
POST	/produtos	Cadastra produto	201
DELETE	/produtos/	Inativa produto	200
POST	/produtos/entrada	Registra entrada	200
POST	/produtos/saida	Registra saída	200
GET	/produtos/saldo	Consulta saldo	200
GET	/produtos/historico	Histórico individual	200
GET	/produtos-estoque-baixo	Lista alertas	200
GET	/dashboard	Conta operações de entrada/saída	200
GET	/relatorio/estoque	Inventário atual	200
GET	/relatorio/movimentacoes	Histórico geral	200

400 Bad Request: JSON ausente, dados inválidos, categoria inexistente/duplicada ou estoque insuficiente.

404 Not Found: produto inexistente ou inativo.

JSON: formato textual usado para trocar dados entre JavaScript e Python. jsonify converte listas e dicionários em resposta JSON.

CORS: está habilitado para permitir requisições do frontend. Como o Flask atualmente entrega o próprio frontend, ele é menos necessário, mas ajuda durante desenvolvimento em origens diferentes.

8. Relatórios e dashboard

- Inventário: id, nome, quantidade, preço, valor_total, estoque_minimo e abaixo_do_minimo.
- valor_total = quantidade × preço, arredondado para duas casas.
- Movimentações: tipo, quantidade, observação, data, saldo anterior e saldo atual.
- Ordenação: data_hora DESC e id DESC, para desempatar operações no mesmo segundo.
- Dashboard: entradas e saídas são COUNT(*), portanto representam número de operações, não total de unidades.

Pegadinha: o histórico geral continua mostrando movimentações de produtos inativados. Já o histórico individual exige que o produto esteja ativo e devolve 404 depois da inativação. Isso pode ser melhorado futuramente com uma rota administrativa.

9. Testes automatizados

O projeto possui 15 testes com unittest. Cada conjunto aponta CAMINHO_BANCO para um arquivo temporário dentro de tests e o remove ao terminar, evitando alterar o estoque.db real.

- test_estoque.py: entrada, saída, saldo, estoque insuficiente, valores inválidos, alerta e inativação.
- test_api.py: rotas, JSON, frontend, dashboard, histórico, códigos HTTP e preservação do histórico.
- test_database.py: atualiza um banco antigo e confirma que os dados permanecem.

```
python -m unittest discover tests -v
```

Por que testar? Para detectar regressões, provar as regras e permitir que branches sejam integradas com mais segurança.

10. Roteiro rápido para demonstrar o backend

1. Inicie com `python -m backend.api` e abra `http://127.0.0.1:5000`.
2. Mostre o cadastro de categoria e explique o `POST /categorias`.
3. Cadastre um produto com quantidade e estoque mínimo.
4. Faça uma entrada e mostre o saldo aumentando.
5. Faça uma saída válida e mostre `saldo_anterior/saldo_atual` no histórico.
6. Tente uma saída maior que o saldo e mostre a mensagem de erro.
7. Mostre o alerta quando `quantidade <= estoque_minimo`.
8. Abra relatórios e explique inventário, valor total e movimentações.
9. Remova um produto e explique que foi inativado; o histórico geral permanece.
10. Finalize citando SQLite, migração e os 15 testes.

11. Limitações atuais e melhorias futuras

Limitação atual	Resposta madura para a banca / melhoria
Sem autenticação	O escopo acadêmico focou estoque; produção teria login, autorização e sessões/tokens.
CORS liberado	Em produção restringiríamos às origens conhecidas.
<code>debug=True</code>	É adequado apenas ao desenvolvimento; produção usaria debug desligado e servidor WSGI.
Preço em REAL	Para maior precisão monetária usaríamos centavos inteiros ou Decimal.
Sem edição PUT/PATCH	Poderíamos criar rota para atualizar produto e validar os mesmos campos.
Campos SKU/fornecedor não expostos	A modelagem está preparada; falta incluir no formulário e na API.
Quantidade inicial sem movimentação	Para auditoria completa, criaríamos uma ENTRADA INICIAL.
SQLite	É ótimo para projeto pequeno; alta concorrência poderia exigir PostgreSQL/MySQL.
Sem paginação/filtros	Relatórios grandes deveriam ter filtros por data, produto, tipo e páginas.

12. Perguntas gerais sobre o projeto

Treine respostas curtas. Se pedirem mais detalhes, complemente com um exemplo do código ou da demonstração.

PERGUNTA	Qual é a função do backend?
RESPOSTA	Receber requisições, validar regras, manipular o SQLite e devolver dados ou erros em JSON.
PERGUNTA	Quais requisitos obrigatórios foram atendidos?
RESPOSTA	Cadastro de produtos/categorias, entrada/saída, saldo automático, estoque mínimo, relatórios e persistência.
PERGUNTA	Qual foi a divisão do trabalho?
RESPOSTA	Edypho trabalhou no backend/API e integração; Lucas no banco, modelagem, saldos e seed; Nicollas no frontend. A equipe revisou o fluxo integrado.
PERGUNTA	Por que escolheram Python?
RESPOSTA	Era uma tecnologia adequada ao conteúdo estudado, tem sintaxe acessível e integração simples com SQLite e Flask.
PERGUNTA	Por que Flask?
RESPOSTA	É leve e suficiente para criar rotas HTTP/JSON sem adicionar complexidade desnecessária ao projeto.
PERGUNTA	Por que SQLite?
RESPOSTA	É simples, local, não exige servidor e atende bem ao volume e à concorrência de um projeto acadêmico.
PERGUNTA	O frontend é Java?
RESPOSTA	Não. O frontend usa JavaScript. Java e JavaScript são linguagens diferentes.
PERGUNTA	Como as partes se comunicam?
RESPOSTA	O JavaScript usa fetch para chamar rotas Flask; a API responde JSON; o backend acessa o SQLite.
PERGUNTA	Onde os dados ficam salvos?
RESPOSTA	No arquivo estoque.db na raiz do projeto.
PERGUNTA	Se fechar o sistema, os dados somem?
RESPOSTA	Não. SQLite fornece persistência em disco; os dados continuam no estoque.db.

13. Perguntas sobre Flask, API e HTTP

Treine respostas curtas. Se pedirem mais detalhes, complemente com um exemplo do código ou da demonstração.

PERGUNTA	O que é uma API?
RESPOSTA	É uma interface com rotas e contratos que permite ao frontend solicitar operações e dados ao backend.
PERGUNTA	O que é uma rota?
RESPOSTA	É a combinação de um caminho, como /produtos, e um método HTTP que executa uma função Python.
PERGUNTA	Diferença entre GET, POST e DELETE?
RESPOSTA	GET consulta; POST envia dados para criar/registrar; DELETE solicita remoção — neste projeto, inativação.
PERGUNTA	Por que POST de cadastro retorna 201?
RESPOSTA	201 Created indica que um novo recurso foi criado com sucesso.
PERGUNTA	Por que produto inexistente retorna 404?
RESPOSTA	Porque o recurso solicitado não foi encontrado entre os produtos ativos.
PERGUNTA	Por que estoque insuficiente retorna 400 e não 500?
RESPOSTA	É um erro esperado da entrada/regra de negócio, não uma falha inesperada do servidor.
PERGUNTA	O que request.get_json(silent=True) faz?
RESPOSTA	Tenta ler o corpo JSON sem lançar erro automático; o sistema então valida se recebeu um dicionário.
PERGUNTA	Para que serve jsonify?
RESPOSTA	Converte dados Python em uma resposta HTTP JSON com tipo de conteúdo adequado.
PERGUNTA	O que é CORS?
RESPOSTA	Uma regra do navegador para chamadas entre origens diferentes. Flask-CORS libera essas chamadas durante o desenvolvimento.
PERGUNTA	A API e o frontend usam a mesma porta?
RESPOSTA	Sim, atualmente o Flask serve ambos em 127.0.0.1:5000.
PERGUNTA	O que significa ?
RESPOSTA	É um parâmetro de rota que o Flask converte para inteiro antes de chamar a função.

PERGUNTA	Como a API trata erros?
RESPOSTA	Exceções específicas possuem error handlers que devolvem JSON e status 400 ou 404.

14. Perguntas sobre banco de dados e SQL

Treine respostas curtas. Se pedirem mais detalhes, complemente com um exemplo do código ou da demonstração.

PERGUNTA	Quais são as tabelas principais?
RESPOSTA	categorias, produtos e movimentacoes.
PERGUNTA	O que é chave primária?
RESPOSTA	Campo que identifica unicamente cada registro; no projeto é o id autoincremental.
PERGUNTA	O que é chave estrangeira?
RESPOSTA	Campo que referencia outra tabela, como categoria_id e produto_id.
PERGUNTA	Qual a cardinalidade?
RESPOSTA	Uma categoria para muitos produtos; um produto para muitas movimentações.
PERGUNTA	Para que serve AUTOINCREMENT?
RESPOSTA	Gera automaticamente um novo ID inteiro para cada registro.
PERGUNTA	Por que nome de categoria é UNIQUE?
RESPOSTA	Para impedir duas categorias com o mesmo nome no banco.
PERGUNTA	O que significa NOT NULL?
RESPOSTA	A coluna precisa receber um valor e não pode ficar nula.
PERGUNTA	O que faz PRAGMA foreign_keys = ON?
RESPOSTA	Ativa a fiscalização de chaves estrangeiras naquela conexão SQLite.
PERGUNTA	Por que usar sqlite3.Row?
RESPOSTA	Para acessar as colunas por nome, deixando o código mais claro.
PERGUNTA	Como evitam SQL Injection?
RESPOSTA	Os valores são enviados por placeholders ?, separados da string SQL.
PERGUNTA	Existe f-string em SQL no projeto. É perigoso?
RESPOSTA	Na migração, tabela e coluna vêm somente de constantes internas, não do usuário. Valores externos continuam parametrizados.

PERGUNTA	O que é commit?
RESPOSTA	Confirma e grava permanentemente as alterações da transação.
PERGUNTA	O que é rollback?
RESPOSTA	Desfaz as alterações ainda não confirmadas quando ocorre um erro.
PERGUNTA	O que é uma transação?
RESPOSTA	Grupo de operações tratado como uma unidade: todas funcionam ou todas são desfeitas.
PERGUNTA	Como o banco antigo é atualizado?
RESPOSTA	O sistema consulta PRAGMA table_info e executa ALTER TABLE somente para colunas ausentes.
PERGUNTA	A migração apaga dados?
RESPOSTA	Não. Ela adiciona colunas e há um teste que confirma a preservação de uma categoria antiga.

15. Perguntas sobre regras e movimentações

Treine respostas curtas. Se pedirem mais detalhes, complemente com um exemplo do código ou da demonstração.

PERGUNTA	Como é calculada uma entrada?
RESPOSTA	<code>saldo_atual = saldo_anterior + quantidade.</code>
PERGUNTA	Como é calculada uma saída?
RESPOSTA	<code>saldo_atual = saldo_anterior - quantidade</code> , desde que exista saldo suficiente.
PERGUNTA	O que acontece em uma saída maior que o saldo?
RESPOSTA	É lançada <code>EstoqueInsuficienteError</code> , ocorre rollback e nada é gravado.
PERGUNTA	Por que salvar saldo anterior e atual?
RESPOSTA	Para auditoria: permite saber exatamente como cada movimentação alterou o estoque.
PERGUNTA	Por que quantidade precisa ser inteira?
RESPOSTA	O estoque foi modelado em unidades discretas; decimal não faz sentido para os produtos deste escopo.
PERGUNTA	Por que bool é rejeitado se Python considera bool um int?
RESPOSTA	Porque <code>True</code> e <code>False</code> não devem ser aceitos como 1 e 0 em campos numéricos de estoque.
PERGUNTA	Qual é a regra do estoque mínimo?
RESPOSTA	O produto entra em alerta quando quantidade é menor ou igual ao <code>estoque_minimo</code> .
PERGUNTA	O alerta envia notificação?
RESPOSTA	Não. É um alerta lógico: o backend lista os produtos e o frontend destaca visualmente.
PERGUNTA	Por que não apagam o produto?
RESPOSTA	Para não perder a ligação e o histórico das movimentações. O campo ativo passa para 0.
PERGUNTA	O que acontece ao buscar produto inativo?
RESPOSTA	As consultas normais filtram <code>ativo = 1</code> , então ele é tratado como não encontrado.
PERGUNTA	O histórico é realmente preservado?
RESPOSTA	Sim. As linhas de <code>movimentacoes</code> continuam no banco e aparecem no relatório geral.

PERGUNTA	Por que strip() é usado?
RESPOSTA	Remove espaços nas extremidades e impede nome formado apenas por espaços.

16. Perguntas sobre relatórios, testes e Git

Treine respostas curtas. Se pedirem mais detalhes, complemente com um exemplo do código ou da demonstração.

PERGUNTA	Como calculam o valor do inventário?
RESPOSTA	Quantidade multiplicada pelo preço unitário, com round para duas casas.
PERGUNTA	O dashboard soma unidades?
RESPOSTA	Não. COUNT(*) conta quantas operações de entrada e saída foram registradas.
PERGUNTA	Por que ordenar por data e ID?
RESPOSTA	Duas movimentações podem ocorrer no mesmo segundo; o ID desempata a ordem.
PERGUNTA	Quanto testes existem?
RESPOSTA	Atualmente são 15 testes automatizados.
PERGUNTA	Os testes usam o banco real?
RESPOSTA	Não. Eles alteram CAMINHO_BANCO para arquivos temporários e removem esses arquivos no final.
PERGUNTA	O que é teste de regressão?
RESPOSTA	Um teste que garante que algo que já funcionava continue funcionando após mudanças.
PERGUNTA	O que o teste de migração prova?
RESPOSTA	Que uma estrutura antiga recebe colunas novas sem perder o registro existente.
PERGUNTA	Por que usaram uma branch de integração?
RESPOSTA	Para resolver conflitos e validar a combinação das contribuições antes de atualizar a main.
PERGUNTA	O que é merge?
RESPOSTA	Operação que combina os históricos e alterações de branches diferentes.
PERGUNTA	Como verificaram o merge?
RESPOSTA	Executaram compilação, validação do JavaScript, seed e os 15 testes antes do push.
PERGUNTA	O que é seed?
RESPOSTA	Script que cria dados conhecidos para demonstração e testes manuais.

PERGUNTA	Executar o seed duplica dados?
RESPOSTA	Não duplica as movimentações do seed porque elas recebem uma tag e o script verifica se já foi executado.

17. Perguntas difíceis e respostas honestas

Treine respostas curtas. Se pedirem mais detalhes, complemente com um exemplo do código ou da demonstração.

PERGUNTA	O sistema está pronto para produção?
RESPOSTA	Não totalmente. Para produção seriam necessários autenticação, configuração segura, logs, servidor WSGI, restrição de CORS e banco mais robusto conforme a carga.
PERGUNTA	Por que debug=True é um problema?
RESPOSTA	O modo debug expõe informações internas e reinicia o servidor; deve ser desligado fora do desenvolvimento.
PERGUNTA	REAL é ideal para dinheiro?
RESPOSTA	Não é o mais preciso por causa de ponto flutuante. Uma evolução seria armazenar centavos inteiros ou usar Decimal.
PERGUNTA	O que aconteceria com muitos usuários simultâneos?
RESPOSTA	SQLite pode limitar escritas concorrentes. Para maior escala usaríamos PostgreSQL/MySQL e revisariamos concorrência/transações.
PERGUNTA	Existe login de administrador?
RESPOSTA	A interface mostra Administrador, mas não há autenticação real. Isso ficou fora do escopo obrigatório.
PERGUNTA	Dá para editar um produto?
RESPOSTA	Ainda não há PUT/PATCH. Seria uma melhoria com validação e uma rota de atualização.
PERGUNTA	Dá para excluir categoria?
RESPOSTA	O banco possui campo ativo, mas a API atual não expõe uma rota de inativação de categoria.
PERGUNTA	Por que sku e fornecedor não aparecem?
RESPOSTA	Esses campos foram adicionados à modelagem, porém ainda não estão conectados ao cadastro e às telas.
PERGUNTA	A quantidade inicial aparece no histórico?
RESPOSTA	Não. Hoje ela entra diretamente no cadastro. Para auditoria completa, criaríamos uma movimentação ENTRADA INICIAL.
PERGUNTA	Há risco de duas saídas simultâneas?
RESPOSTA	Para o escopo local é aceitável, mas alta concorrência exigiria transação mais rígida, atualização condicional/lock e possivelmente outro SGBD.

PERGUNTA	Por que não usaram ORM?
RESPOSTA	Para um projeto iniciante, SQL direto deixa a modelagem e as consultas visíveis. Um ORM seria possível, mas adicionaria abstração.
PERGUNTA	Por que não adotaram services/repositories?
RESPOSTA	A versão final priorizou módulos funcionais simples, evitando duas arquiteturas paralelas. A separação por responsabilidade continua existindo entre arquivos.

18. Resumo de bolso — leia antes da apresentação

Pergunta-chave	Resposta em uma frase
Stack	Python + Flask + SQLite no backend; HTML/CSS/JavaScript no frontend.
Comunicação	fetch envia HTTP/JSON; Flask valida, acessa o banco e devolve JSON.
Saldo	Entrada soma, saída subtrai e nunca pode deixar quantidade negativa.
Transação	INSERT do histórico e UPDATE do saldo são confirmados juntos.
Estoque baixo	quantidade <= estoque_minimo.
Remoção	UPDATE ativo = 0; o histórico não é apagado.
Banco	Três tabelas, chaves estrangeiras e migração sem perda de dados.
Erros	400 para dados/regra inválida; 404 para produto não encontrado/inativo.
Testes	15 testes com bancos temporários.
Principal limitação	Sem autenticação e ainda não preparado para alta concorrência/produção.

Fechamento recomendado: “O projeto atende aos requisitos propostos e mantém uma estrutura simples, adequada ao nosso nível acadêmico. Também identificamos melhorias futuras, como autenticação, edição de produtos e maior robustez para produção.”